

Automaton Specification Language Documentation

Automaton Emulator

1 Introduction

The Automaton Specification Language (ASL) is a domain-specific language designed to define finite automata for simulation. It supports both Deterministic Finite Automata (DFA) and Non-deterministic Finite Automata (NFA).

2 File Structure

An ASL file is divided into sections, each denoted by a header in square brackets. Sections can appear in any order, though the following order is conventional:

1. `[alphabet]`
2. `[states]`
3. `[initial_state]`
4. `[accept_states]`
5. `[transitions]`
6. `[settings]` (optional)

3 Sections

3.1 Alphabet

The `[alphabet]` section defines the set of symbols accepted by the automaton. Symbols are space-separated and can span multiple lines.

3.2 States

The `[states]` section lists all possible states in the automaton. Like the alphabet, states are space-separated.

3.2.1 State Ranges

To define a sequence of states with a common prefix and a numerical range, you can use the range syntax: `prefix[start-end]`.

```
1 q[0-15] # Expands to q0 q1 q2 ... q15
```

3.3 Initial State

The `[initial_state]` section must contain exactly one state from the defined states list. This represents the starting point of the simulation.

3.4 Accept States

The `[accept_states]` section lists the states that lead to an 'Accepted' result if the simulation ends in one of them. State ranges are also supported here.

3.5 Transitions

The `[transitions]` section defines the state transition function.

3.5.1 Wildcards

The asterisk `*` can be used as a wildcard for input symbols, representing all symbols in the defined alphabet.

```
1 q0, * -> q1 # Transition to q1 on any symbol from the alphabet
```

In DFAs, a specific transition defined for a symbol will take precedence over a wildcard transition if it appears later in the file.

3.6 Settings

The `[settings]` section allows for optional behavioral modifications of the automaton. Settings are listed line-by-line or space-separated.

3.6.1 RemainInCurrentStateOnUndefinedTransition

When this setting is enabled, the automaton will not 'Hang' if it encounters a symbol for which no transition is defined for the current state. Instead, it will remain in its current state and continue processing the rest of the input string. This is particularly useful for building text-based games or parsers where some inputs should simply be ignored or keep the state unchanged.

4 Transition Syntax

4.1 DFA Transitions

For Deterministic Finite Automata, each transition must follow the strict format:

```
1 source_state, symbol -> next_state
```

4.2 NFA Transitions

Non-deterministic Finite Automata support additional syntax for flexibility:

Single Transition:

```
1 q0, a -> q1
```

Multiple Next States:

```
1 q0, a -> q1, q2
```

Multiple Input Symbols:

```
1 q0, (a, b) -> q1
```

Wildcard Transition:

```
1 q0, * -> q1
```

5 Lexical Rules

- **Comments:** Any text following a `#` symbol on a line is ignored.
- **Whitespace:** Leading and trailing whitespace on lines is ignored. Empty lines are ignored.
- **Case Sensitivity:** Section headers are case-insensitive (`[ALPHABET]` is the same as `[alphabet]`).

6 Example

```
1 [alphabet]
2 a b
3
4 [states]
5 s0 s1
6
7 [initial_state]
8 s0
9
10 [accept_states]
11 s1
12
13 [transitions]
14 s0, a -> s1
15 s1, b -> s1
16 s1, a -> s0
```

Listing 1: Example DFA file

```
1 [alphabet]
2 0 1
3
4 [states]
5 q[0-2]
6
7 [initial_state]
8 q0
9
10 [accept_states]
11 q2
12
13 [transitions]
14 q0, * -> q0
15 q0, 1 -> q1
16 q1, (0, 1) -> q2
```

Listing 2: Example NFA file with ranges and wildcards